

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

BUILDING A FILE SHARING APPLICATION

2nd Progress Report: System & Protocol Design

Người thực hiện:

Cao Van Dong
Luong Ngoc Vu Long
Tran Le Minh

ONE LOVE. ONE FUTURE.

1. Application Introduction & Problem Statement

Motivation & Collaboration Needs

- **Structured Team Workspaces**: Projects require dedicated, group-isolated folders to organize code, documents, and assets without cross-group leakage.
- **Preventing Accidental Loss**: Public/open shared drives suffer from unauthorized deletions and unintended overwrites by arbitrary members.
- **Network Fragility**: Incomplete network transfers leave corrupt, unusable ghost files on the server without transactional staging.

Proposed System: Group File Sharing

- **Client-Server Architecture**: Each group has a shared folder on the server. Files remain available when uploaders disconnect, while the server is reachable.
- **Group Permissions**: Members can upload, download, and create subfolders; **only Group Admins** can rename or delete files and folders.
- **File Transfer**: Custom LTV frames over TCP, bounded file chunks, and SHA-256 verification before publishing uploads.

2. System Functionality & Use Case Model

System Actors & Scope

- **Guest (Unauthenticated)**: Registers account (`AUTH_REGISTER`) and signs in via credentials (`AUTH_LOGIN`).
- **Regular Group Member**: Joins groups, browses directory trees (`DIR_LIST`), uploads files, downloads assets, and creates subfolders.
- **Group Admin (Creator)**: Reviews join requests (`GROUP_JOIN_REVIEW`); holds exclusive authority to rename and delete files and folders.

Core Collaboration Use Cases

- **Identity & Session (UC1)**: Account registration, credential authentication, and persistent session binding.
- **Group Lifecycle (UC2–UC3)**: Create group (auto-assigns Admin), browse groups, and review pending member requests.
- **Workspace Navigation (UC4, UC7)**: Hierarchical subfolder navigation and directory creation.
- **Streamed File Transfer (UC5–UC6)**: Chunked uploads and downloads with SHA-256 verification. Interrupted transfers restart in v1.
- **Administrative Protection (UC8)**: Exclusive admin controls for folder/file deletion and renaming.
- **Real-Time Notifications (UC9)**: Server push events for member joins, uploads, and modifications.

Group-Scoped Security Policy

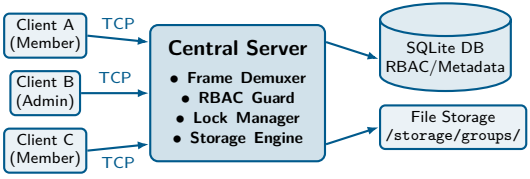
Permissions evaluated dynamically per command on (`userId`, `groupId`, `role`).

Operation	Guest	Non-Member	Member	Group Admin	Security Policy / Enforcement Rule
`AUTH_REGISTER` / `AUTH_LOGIN`	✓	×	×	×	Available before login
`GROUP_CREATE`	×	✓	✓	✓	Authenticated user becomes Admin of new group
`GROUP_JOIN_REQ`	×	✓	×	×	Non-members can submit pending request
`GROUP_JOIN_REVIEW`	×	×	×	✓	**Admin only**: Approve or reject applicants
`DIR_LIST` / `FILE_INFO`	×	×	✓	✓	Restricted to approved group members
Download files	×	×	✓	✓	Read access granted to all members
Upload / `DIR_CREATE`	×	×	✓	✓	New items only; no overwrite
`FILE_RENAME` / `DIR_RENAME`	×	×	×	✓	**Admin only**
`FILE_DELETE` / `DIR_DELETE`	×	×	×	✓	**Admin only**

Paradigm Evaluation

- **Pure P2P (Discarded)**: NAT/firewall traversal barriers; offline peers cause file unavailability; complex RBAC consensus.
- **Hybrid (Discarded)**: High signaling overhead; asymmetric bandwidth; weak consistency during renames.
- **Client-Server (Selected Architecture)**:
  - **Authoritative RBAC**: Single source of truth for access control.
  - **Independent of Peers**: Files stay on the server when members disconnect.
  - **Tradeoff**: The server must be reachable and has finite storage and bandwidth.

High-Level System Topology



5. Server Engine & Multi-Tier Architecture

Server Component Tiers

- **Connection Layer**: One worker per connection processes incoming frames in order. Outgoing frames are serialized per socket.
- **Session & RBAC**: Login binds a user to the connection; each group command checks membership and role.
- **Concurrency**: One read/write lock per group. Members can read together; changes to shared content are exclusive.
- **Dual Storage Layer**: Relational SQLite (accounts, RBAC, metadata) + partitioned file tree.

Server Processing Pipeline

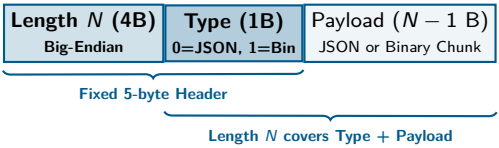
1. **TCP Stream Demuxer**: Accumulates bytes; verifies 5-byte header; extracts frame
2. **Session & RBAC**: Checks the connection identity and target group permissions
3. **Lock Arbitration**: Takes a group read or write lock when accessing shared content
4. **Command Execution**: Queries metadata, streams bytes, or commits a change
5. **Response & Events**: Replies to caller; sends events to eligible users

6. Protocol Design: Transport Choice & TCP Framing

Transport Layer: TCP Byte-Stream

- **Why TCP**: Reliable, ordered byte stream with flow control. The application still handles connection failures.
- **Boundary Challenge**: A socket read can return part of a frame or several frames.
- **LTV Framing**: Read the 4-byte length first. Close the connection if N is outside 1 to 1,048,576; otherwise read exactly N more bytes.

Length-Type-Value (LTV) Frame Layout



- $N = 1 +$ Payload size in bytes.
- **Type 0x00**: UTF-8 JSON Control / Event Message.
- **Type 0x01**: Raw Binary File Chunk with offset header.

7. Framing Specification: Type 0x00 vs Type 0x01

Type 0x00: JSON Control / Event Frame

Used for authentication, directory navigation, transfer control, and push event broadcasting.

Length N (4B)	**Type = 0x00 (1B)**	UTF-8 JSON Payload (Request / Response / Event envelope)

Type 0x01: Raw Binary File Chunk Frame

Carries file bytes directly, avoiding Base64 encoding.

Length (4B)	**Type = 0x01**	Transfer UUID (16B)	Offset (8B)	ChunkLen (4B)	**Raw File Bytes (ChunkLen)**

- **Transfer UUID (16 Bytes)**: Identifies the active upload/download staging session.
- **Offset (8 Bytes)**: Must match the next expected byte; starts at zero.
- **ChunkLen (4 Bytes)**: 1 to 32 KiB of raw bytes; $N = 29 + \texttt{ChunkLen}$. Integers use big-endian encoding.

8. Message Envelope Schema

Client Request Envelope

```
{
  "v": 1, "kind": "REQUEST",
  "reqId": "r17",
  "cmd": "FILE_UPLOAD_INIT",
  "payload": {
    "groupId": 4, "path": "/reports/final.pdf",
    "fileSize": 1048576, "sha256": "e3b0c442..."
  }
}
```

Server Push Event (Unsolicited)

```
{
  "v": 1, "kind": "EVENT",
  "event": "GROUP_FILE_ADDED",
  "payload": {
    "groupId": 4, "path": "/reports/final.pdf",
    "byUser": "dongcv"
  }
}
```

Server Response Envelope

```
{
  "v": 1, "kind": "RESPONSE",
  "reqId": "r17",
  "cmd": "FILE_UPLOAD_INIT",
  "ok": true,
  "payload": {
    "transferId": "9b1deb4d-3b7d-4bad...",
    "chunkSize": 32768, "nextOffset": 0
  }
}
```

```
{
  "v": 1, "kind": "RESPONSE",
  "reqId": "r18",
  "cmd": "DIR_DELETE",
  "ok": false,
  "error": {
    "code": "FORBIDDEN_NOT_ADMIN",
    "message": "Only Group Admin can delete."
  }
}
```

Authentication & Group Membership

v1 design: send credentials over TLS; store salted password hashes on the server.

Command	Caller Role	Key Request Parameters	Key Response / Event Payload
`AUTH_REGISTER`	Unauthenticated	`username`, `password`, `fullName`	`userId`; account created
`AUTH_LOGIN`	Unauthenticated	`username`, `password`	`userId`; binds user to connection
`AUTH_LOGOUT`	Authenticated	None	Clears connection identity
`HEARTBEAT_PING`	Any connected	`timestamp`	`pong`, connection health check
`GROUP_CREATE`	Authenticated	`groupName`, `description`	`groupId`, creator becomes **Group Admin**
`GROUP_LIST`	Authenticated	Filter: `"ALL"` or `"MY_GROUPS"`	Array of `[groupId, name, role, adminName]`
`GROUP_JOIN_REQ`	Non-Member	`groupId`, optional `message`	`joinRequestId`; `PENDING`
`GROUP_JOIN_REVIEW`	**Group Admin**	`joinRequestId`, `ACCEPT` or `REJECT`	Records decision; grants access on acceptance only
`GROUP_MEMBERS`	Group Member	`groupId`	Roster of members with roles (`MEMBER`/`ADMIN`)
`GROUP_LEAVE`	Group Member	`groupId`	Revokes access; Admin cannot abandon group

Directory Structure & File Metadata Operations

Command	Caller Role	Key Request Parameters	Key Response / Error Behavior
`DIR_LIST`	Member / Admin	`groupId`, `dirPath`	Entries: `[name, isDir, size, modified, author]`
`DIR_CREATE`	Member / Admin	`groupId`, `parentPath`, `dirName`	Creates subfolder; rejects a duplicate name
`DIR_RENAME`	**Admin Only**	`groupId`, `path`, `newName`	Renames folder; denies non-admin requests
`DIR_DELETE`	**Admin Only**	`groupId`, `path`, `recursive`	Recursively deletes directory; checks write lock
`FILE_INFO`	Member / Admin	`groupId`, `filePath`	Returns size, SHA-256 hash, author, timestamp
`FILE_RENAME`	**Admin Only**	`groupId`, `filePath`, `newName`	Renames file; denies non-admin requests
`FILE_DELETE`	**Admin Only**	`groupId`, `filePath`	Removes file and notifies group members

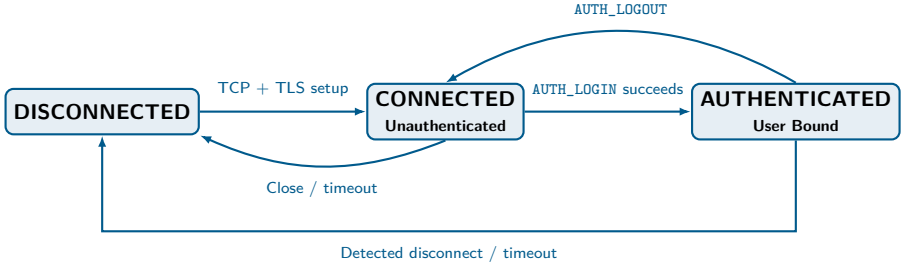
Staged Upload, Resilient Streaming, and Download Commands

Command	Caller Role	Key Request Parameters	Key Response / Protocol Behavior
`FILE_UPLOAD_INIT`	Member / Admin	`groupId`, `path`, `fileSize`, `sha256`	Allocates `.tmp` file; returns `transferId`, `chunkSize`
`Type 0x01 Chunk`	Member / Admin	`UUID`, `offset`, `chunkLen`, `rawBytes`	Ingests binary chunk; advances staging offset
`FILE_UPLOAD_FINISH`	Member / Admin	`transferId`	Verifies size and hash; commits file and metadata before success
`FILE_UPLOAD_ABORT`	Member / Admin	`transferId`	Discards staging file and ends transfer
`FILE_DOWNLOAD_INIT`	Member / Admin	`groupId`, `filePath`	Takes group read lock; replies with `transferId`, `fileSize`, `sha256`
`Type 0x01 Stream`	Server → Client	`UUID`, `offset`, `chunkLen`, `rawBytes`	Streams from byte zero; releases lock after last write or failure

Client verifies downloaded size and hash before saving the final file. No application ACK is needed.

Connection Lifecycle States & Event Transitions

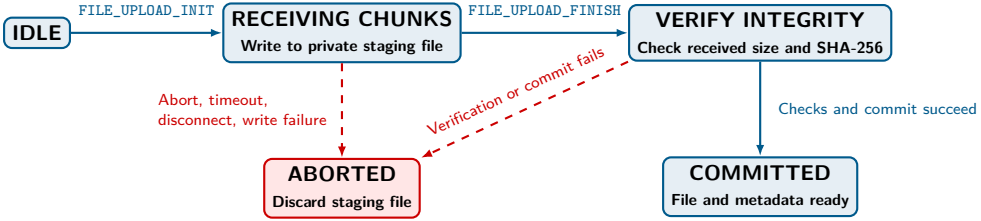
Login binds a user to the connection. Each group command identifies its target group.



- A failed login leaves the connection unauthenticated.
- v1 allows one active transfer per connection and sets a deadline for stalled transfers.
- On disconnect or timeout, the worker clears the session, discards partial uploads, and frees locks.

Staged File Upload Lifecycle: Preventing Corrupted Transfers

Directly resolves the challenge of broken TCP connections leaving partial ghost files on disk.

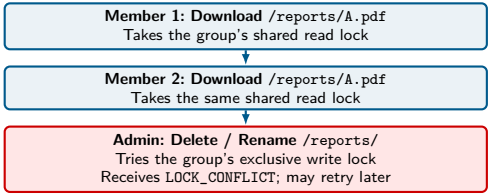


- **Receiving**: Each accepted chunk advances the expected offset; receiving all bytes does not publish the file.
- **Commit**: Recheck permissions and destination under the group write lock. Move the file on the same filesystem and record its metadata before replying.
- **Recovery**: Failed uploads stay out of group listings. On startup, reconcile staging files and files moved before their metadata was recorded.

Server Pipeline Workflow

1. **Decode**: Check length and type. Close on invalid framing.
2. **Validate**: Check command fields and state. For chunks, check transfer ownership, expected offset, and remaining size.
3. **Authorize**: Check the user's role in the target group. Reject paths outside that group's root.
4. **Execute**: Take the required group lock and perform the operation.
5. **Reply**: Return a result or error. Send change events after success.

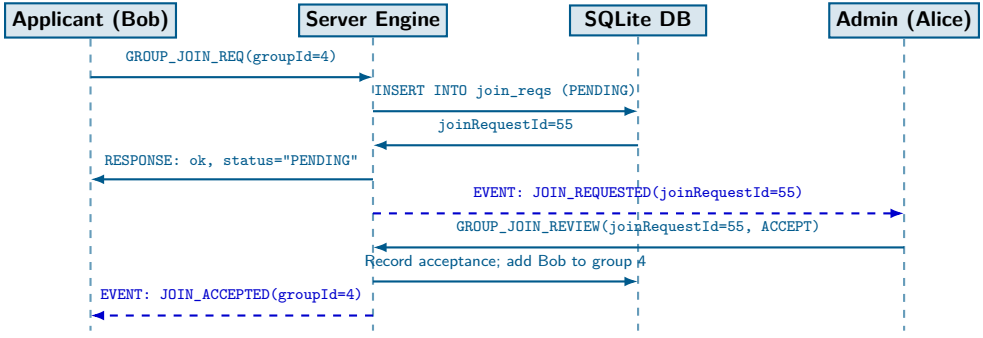
Locking Architecture: Solving Race Conditions



- The download worker releases its lock after the last write or on failure. A group lock can also block writes to unrelated folders in that group.

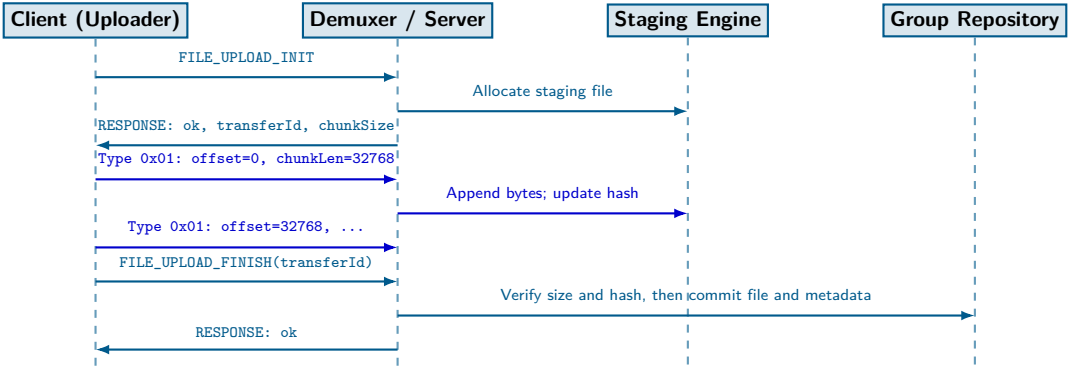
15. Sequence: Group Join Request & Admin Review

Interaction: Applicant Submits Join Request → Admin Approves

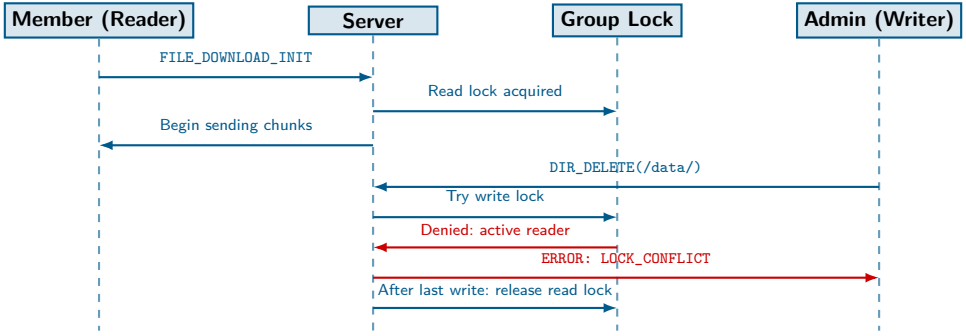


16. Sequence: Chunked Upload & Commit

Interaction: Receive Chunks, Verify, Then Publish



Conflict Arbitration: Active Download vs Admin Delete Command



18. Implementation Roadmap & Next Milestones

Phase 1 to 3: Core Platform & RBAC

- **Phase 1: Design Review (W1–2)**: Use cases, RBAC matrix, LTV framing, command catalog. (**Status: Done**)
- **Phase 2: Core Socket Engine (W3–4)**: TCP workers, TLS, frame decoder, SQLite schema, authentication.
- **Phase 3: Group Management (W5–6)**: Group CRUD, join request review, subfolder CRUD, admin delete/rename.

Phase 4 & 5: Streaming & Client GUI

- **Phase 4: Streaming & Staging (W7–8)**:
  - Type 0x01 chunk streaming (32 KiB) & `.tmp` staging.
  - Streaming SHA-256 verification and file commit.
  - Timeout and disconnect cleanup; restart failed transfers.
- **Phase 5: JavaFX GUI & QA (W9–10)**:
  - Desktop client UI with transfer progress bars.
  - Multi-client stress testing & course defense.

19. Design Summary & Feedback

Core Contributions in Report 02

- **LTV Framing Protocol**: 5-byte header resolves TCP stream boundary ambiguity cleanly.
- **Multiplexed Streaming**: JSON control frames and raw binary chunks multiplexed over one persistent TCP socket.
- **Group Permissions**: Members contribute files; admins alone rename or delete them.
- **Upload Staging**: Verify size and SHA-256 before adding a file to the group listing.

Decisions for Feedback

- Is one lock per group sufficient for the first version?
- Should interrupted transfers restart in v1, or is resume required?
- Next: revise the protocol based on feedback, then test split frames, forbidden actions, and interrupted transfers.



THANK YOU !